

The overall structure is based around classes having their own individual roles inside the game. The class “Game” is the overarching class of the code. “Game” contains the player and map; the role of “Game” is to update and control the flow of gameplay. The class “Player” is used as a representative of the user’s character in the game, it holds their health (first way to die), their hunger (2nd way to die), contains their inventory, and tracks any other player-related data. The class “WorldMap” is used to hold all of our locations and organize them as a collective. The “WorldMap” will be used to carry out the movement of the player. The class “Location” holds all the data for a location the player is in, it is responsible for handling the player interaction with these environments. We implement generalization in the creation of Locations to satisfy the principles of SOLID and DRY when creating them to the best of our abilities. The “NPC” class is designed to be interactable by the player and each “NPC” is associated with a particular Location by being placed in a vector that is an attribute of said location. “Inventory” is responsible for handling all storage and transactions involving “Items”. Items are objects that can be used by the player, there are generalizations in order to easily create different items with unique functionalities. “Materials” will only be used for transactions, “Food” can additionally be used to restore health, and “Weapons” can be used to improve survivability in an encounter. “CraftingRecipe” is designed to assist inventory with transactions, it stores what the price of getting an item is in a transaction and can then carry out said transaction when capable. “UserInput” is responsible for taking inputs from the user and returning them into the program. “ScreenDisplay” is responsible for sending the user information from the program, we created the display functions to be as versatile as possible. The third way to lose in our game is by killing an NPC. The biggest changes made between design and basic implementation were the addition of getter and setter functions which were largely omitted from our original designs.

Our design will satisfy DRY because we make functions to carry out tasks and we make new classes in order to prevent repeating ourselves when implementing. As a specific example, instead of designing each Player, Mob, NPC, etc. to have their own vector of items, we create a class that handles that and then give each class an object of Inventory. By having this class, we only need to make the associated functions once. We satisfy S (single responsibility) in SOLID since each class is only given 1 role, for example, “UserInput” handles inputs from the user, Game coordinates the code without having excess laid, “ScreenDisplay” is responsible for outputting to the User, Item is responsible for how it is used, etc. We satisfy O(open/closed) by having the ability to add onto our code, we can non-destructively add new items/locations/npcs without ruining the entire code base. Our design should satisfy L, child classes can stand in for parent classes, since a class like Inventory does not care if it is dealing with a Food or a Material. We satisfy I by not giving classes unnecessary functions, for example, HelpNPC does not need BuyItem, therefore it does not have it.